

Deep Research Chat — /deepchat

Full 5-layer architecture as a dedicated endpoint, built on the Digilab V2 stack

Prepared for: Abdul Rahman | Date: July 8, 2026 | Repo: github.com/Asvix-04/V2 (Backend_chatbot)

1. Two Separate Endpoints

Instead of one endpoint with a mode flag, Deep Research gets its own dedicated route. Cleaner separation, independent scaling/rate-limiting later, and the frontend simply calls a different URL for the Deep Research toggle instead of changing a request field.

Endpoint	Pipeline	Notes
POST /chat	Single-hop RAG (unchanged): retrieve → sufficiency gate → answer or refuse.	Existing code, untouched. api_server.py, chatbot.py stay exactly as they are.
POST /deepchat	Full 5-layer pipeline: Orchestrator → Worker Agents → Verifier → Retrieval (corpus + web) → Synthesizer.	New route + new engine module. Reuses the existing retriever, LLM client, and validator as building blocks.

2. How /deepchat Works, Step by Step

Before the technical layer breakdown, here's the flow in plain language — what actually happens between a question coming in and an answer going out.

1. Question comes in

The user sends a question to /deepchat instead of /chat. Different door, same building — it shares the retriever, LLM client, and validator already built for /chat.

2. Orchestrator checks: one question, or many?

A quick check looks for words like “compare”, “vs”, or multiple question marks. Simple single-topic questions aren't split — no extra cost. Multi-part questions (“difference between X and Y”) get broken into 2–3 sub-questions.

3. Each sub-question searches the corpus — in parallel

Every sub-question runs through the existing retrieval pipeline at the same time, not one after another: Pinecone searches by meaning, BM25 searches by keyword, Neo4j pulls in related course sections. These parallel retrieval calls are the “Worker” step.

4. Verifier checks: is this actually enough?

For each sub-question, the same sufficiency check already in the codebase runs: does this content really cover the topic, or is it just a passing mention? A weak score means don't guess — move to step 5.

5. If the corpus falls short, go to the web

Only for the sub-questions that failed the check — not the whole question — a web search runs (Tavily). Results come back clearly tagged as internet-sourced, never silently mixed with syllabus material.

6. Synthesizer merges everything into one answer

All verified pieces — some from the syllabus, maybe some from the web — are combined into one coherent, clearly-labeled response.

One line summary: split the question if needed → search the syllabus for each part → check if that's enough → only search the web for the parts that aren't → merge it all into one clearly-labeled answer. The thing that makes /deepchat genuinely different from /chat isn't a smarter model — it's that it doesn't give up when the syllabus falls short. /chat refuses; /deepchat keeps digging.

3. The 5-Layer Architecture — Mapped to /deepchat

Same layer structure as the original Deep Research design (Orchestrator → Worker Agents → Verifier → Retrieval → Synthesizer) — just implemented with your actual stack instead of generic placeholders.

Layer	Job in /deepchat	Implementation
1. Orchestrator / Planner	Reads the question, decides if it's multi-part, splits it into sub-questions, and builds the task list.	One LLM call (Gemini 2.5 Flash) — only triggered when multi-topic phrasing is detected (“vs”, “compare”, multiple “?”).
2. Worker Agents	Executes each sub-question: runs retrieval, gathers evidence.	One worker call per sub-question, run in parallel with asyncio. Each worker calls the shared retriever, not a separate model.
3. Verifier / Citer	Checks each worker's retrieved evidence for grounding/sufficiency before it's allowed into the final answer.	Reuses the existing <code>_validate_content_sufficiency()</code> gate — <code>completeness_score 1–10</code> + <code>is_main_subject</code> check.
4. Retrieval Layer	Supplies grounding content: closed corpus first, web search only if the Verifier rejects the corpus result.	Pinecone (vector) + BM25 (keyword) + Neo4j (graph context) first; Tavily web search as the fallback tier.
5. Synthesizer	Merges every verified sub-answer (corpus-sourced and web-sourced) into one coherent, cited report.	Final LLM call (Gemini/Claude/DeepSeek via the existing <code>UnifiedLLMClient</code>) with inline <code>[Course Material]</code> / <code>[URL]</code> labels.

This is the exact same 5-box flow from the original architecture plan — the only thing that changed is which real tool sits in each box, and that Worker Agents here are parallel retrieval calls rather than separate heavyweight models (your corpus is one domain, so one retrieval pipeline reused per sub-question is more efficient than spinning up different worker models).

4. Models & Services Per Layer

Layer	Model / Service	Access
Orchestrator	Gemini 2.5 Flash (existing default model)	Free tier + paid API
Worker Agents	No separate model — parallel calls into the shared retrieval pipeline (asyncio)	N/A — infra only
Verifier / Citer	Gemini 2.5 Flash (existing _validate_content_sufficiency)	Free tier + paid API
Retrieval — corpus	Pinecone (all-MiniLM-L6-v2 embeddings) + BM25 + Neo4j graph context	Pinecone free tier; BM25/embeddings free & local; Neo4j AuraDB free tier
Retrieval — web fallback	Tavily (search_depth="advanced")	Free tier: 1,000 searches/month
Synthesizer	Gemini 2.5 Flash/Pro, DeepSeek V3.2, or Claude — selectable via existing UnifiedLLMClient	Free (Gemini) up to paid (Claude)

Why Worker Agents don't need their own model here

In a generic multi-domain Deep Research system, Worker Agents often run different specialized models (one for code, one for search, one for math). Here, every sub-question needs the same thing — course-corpus lookup — so the “worker” is really the retrieval pipeline itself, called once per sub-question and run concurrently. This keeps the layer count honest without adding model calls you don't actually need.

5. /deepchat Endpoint Contract

Field	Type	Notes
question	string	Same as /chat
use_history	bool, optional	Same as /chat
model	string, optional	Which synthesizer model to use — same AVAILABLE_MODELS ids as /chat

Field	Type	Notes
answer	string	Final synthesized report
sub_questions	list[string]	What the Orchestrator split the question into
layer_trace	list[object]	Per sub-question: which layer resolved it (corpus vs web) and the Verifier's score — useful for debugging/metrics
sources	list[object]	Corpus citations (existing format, unchanged from /chat)
web_sources	list[object]	{title, url} for any web-sourced sub-answers

layer_trace is the one genuinely new field — it's what lets you show "which layer answered this" in the UI or in a metrics dashboard, and it's cheap to add since every layer already produces this information internally.

6. Free vs. Paid — /deepchat Stack

Layer	Free Path	Paid Path (scale-up)
Orchestrator / Verifier / Synthesizer (LLM)	Gemini 2.5 Flash free tier	Gemini paid tier, or Claude Sonnet/Haiku for higher-quality synthesis
Vector DB	Pinecone Starter (free)	Pinecone Standard — more QPS/storage
Graph DB	Neo4j AuraDB Free	AuraDB Professional
Web search	Tavily free tier (1,000/mo)	Tavily paid, or Serper.dev at higher volume
Worker execution	asyncio (Python stdlib) — always free	N/A

7. Rollout Plan

Recommended sequence

Phase 1: Stand up /deepchat with just Layers 1, 4 (corpus-only), 5

New route, new engine file. Orchestrator splits the question, each sub-question retrieves from the existing corpus pipeline, Synthesizer merges. No Verifier gate or web fallback yet — confirm the route works and produces sensible merged answers for multi-part questions.

Phase 2: Add the Verifier layer

Wire in the existing sufficiency check per sub-question. At this point /deepchat can already refuse sub-questions it can't answer, same as /chat does today — just per sub-question instead of per whole query.

Phase 3: Add the web-search Retrieval fallback

When the Verifier rejects a sub-question's corpus result, call Tavily and label the result as web-sourced. Test explicitly with a question outside the syllabus to confirm the fallback fires and is clearly labeled.

Phase 4: layer_trace + frontend toggle

Add the layer_trace field to the response, reuse metrics_logger.py to log per-layer stats, and wire the frontend to call /deepchat when the Deep Research toggle is on, /chat otherwise.

Source: github.com/Asvix-04/V2, branch updated-frontend-2, path Backend_chatbot, as of July 8, 2026. Web search provider pricing verified as of early July 2026 — verify current rates before committing budget.